

the following reasons:

- Missing and outdated statistics
- Suboptimal heuristics
- Wrong estimation of costs

Spark 3.x has enhanced this process by using runtime data for iteratively improving the query planning and optimisation. The runtime statistics of a prior stage are used to optimise the query plan for subsequent stages. There is a feedback loop that helps to re-plan and re-optimize the execution plan.

Adaptive query execution

(AQE): The query is changed to a logical plan and finally to a physical plan. The concept here is 'reoptimisation'. It takes the data available during the prior stage and reoptimises for subsequent stages. Because of this, the overall query execution is much faster.

AQE can be enabled by setting the SQL configuration, as given below (default false in Spark 3.0):

```
spark.conf.set("spark.sql.adaptive.enabled", true)
```

Dynamically coalescing shuffle

partitions: Spark determines the optimum number of partitions after a shuffle operation. With AQE, Spark uses the default number of partitions, which is 200. This can be enabled by the configuration:

```
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", true)
```

Dynamically switching join

strategies: Broadcast Hash is the best join operation. If one of the data sets is small, Spark can dynamically switch to Broadcast join instead of shuffling large amounts of data across the network.

Dynamically optimising skew

joins: If the data dispersion is not uniform, data will be skewed and there will be a few large partitions.

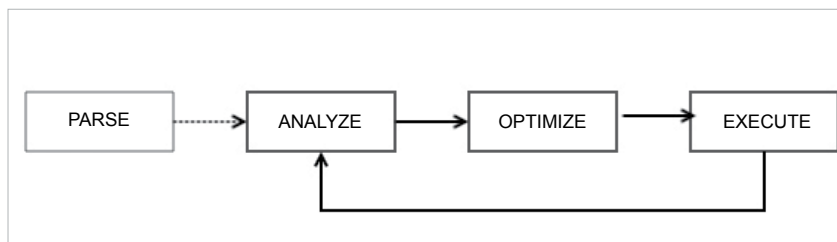


Figure 1: Query planning

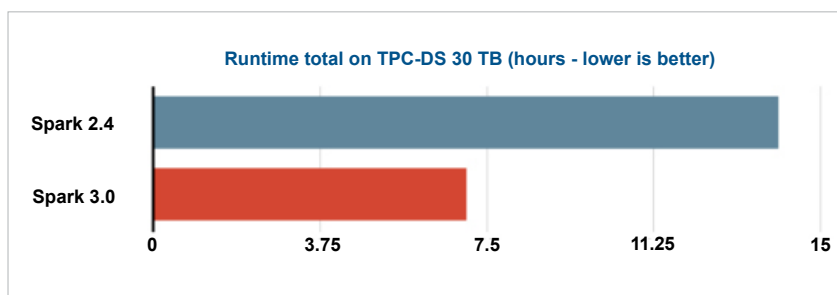


Figure 2: Performance improvement in Spark 3.x (Source: Databricks)

These partitions take up a lot of time. Spark 3.x optimises this by splitting the large partitions into multiple small partitions. This can be enabled by setting:

```
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", true)
```

Other enhancements

In addition, Spark SQL 3.x supports the following.

Dynamic partition pruning: 3.x will only read the partitions that are relevant based on the values from one of the tables. This eliminates the need to parse the big tables.

Join hints: This allows users to specify the join strategy to be used if the user has knowledge of the data. This enhances the query execution process.

ANSI SQL compliant: In the earlier versions of Spark, which are Hive compliant, we could use certain

keywords in the query which would work perfectly fine. However, this is not allowed in Spark SQL 3, which has full ANSI SQL support. For example, 'cast a string to integer' will throw runtime exception. It also supports reserved keywords.

Newer Hadoop, Java and Scala

versions: From Spark 3.0 onwards, Java 11 and Scala 2.12 are supported. Java 11 has better native coordination and garbage correction, which results in better performance. Scala 2.12 exploits new features of Java 8 and is better than 2.11.

Spark 3.x has provided these useful features off-the-shelf instead of developers worrying about them. This will improve the overall performance of Spark significantly. **END** 🐧

Reference

- [Databricks.com](https://databricks.com)

By: Phani Kiran

The author has over 17 years of experience in the IT industry and has worked on Big Data technologies.